Relational Schema & DDL

DBMS Experiential Learning - Level 3 Project | Municipal Civic Grievance Redressal System

1. Relational Schema

Departments

Column	Type	PK	FK	References	Null	Default / Check / Unique
department_id	SERIAL	PK		-	NO	-
department_name	VARCHAR(100)			-	NO	UNIQUE
description	TEXT			-	YES	-
created_at	TIMESTAMP			-	NO	DEFAULT CURRENT_TIMESTAMP

Wards

Column	Type	PK	FK	References	Null	Default / Check / Unique
ward_id	SERIAL	PK		-	NO	-
ward_name	VARCHAR(100)			-	NO	-
ward_number	INTEGER			-	NO	UNIQUE
zone	VARCHAR(50)			-	YES	-
created_at	TIMESTAMP			-	NO	DEFAULT CURRENT_TIMESTAMP

Users

Column	Type	PK	FK	References	Null	Default / Check / Unique
user_id	SERIAL	PK		-	NO	-
name	VARCHAR(100)			-	NO	-
email	VARCHAR(150)			-	NO	UNIQUE
password_hash	VARCHAR(255)			-	NO	-
phone	VARCHAR(15)			-	NO	UNIQUE
role	VARCHAR(20)			-	NO	CHECK (role IN ('citizen','staff','admin'))
department_id	INTEGER		FK	Departments(department_id)	YES	-
ward_id	INTEGER		FK	Wards(ward_id)	YES	-
created_at	TIMESTAMP			-	NO	DEFAULT CURRENT_TIMESTAMP

Table-level CHECK: (role = 'staff' AND department_id IS NOT NULL AND ward_id IS NOT NULL) OR (role = 'admin' AND ward_id IS NOT NULL AND department_id IS NULL) OR (role = 'citizen' AND department_id IS NULL AND ward_id IS NULL)

ComplaintCategories

Column	Type	PK	FK	References	Null	Default / Check / Unique
category_id	SERIAL	PK		-	NO	-
category_name	VARCHAR(100)			-	NO	UNIQUE
department_id	INTEGER		FK	Departments(department_id)	NO	-
sla_hours	INTEGER			-	NO	DEFAULT 72
created_at	TIMESTAMP			-	NO	DEFAULT CURRENT_TIMESTAMP

Complaints

Column	Type	PK	FK	References	Null	Default / Check / Unique
complaint_id	SERIAL	PK		-	NO	-
citizen_id	INTEGER		FK	Users(user_id)	NO	-

Column	Type	PK	FK	References	Null	Default / Check / Unique
category_id	INTEGER		FK	ComplaintCategories(category_id)	NO	-
ward_id	INTEGER		FK	Wards(ward_id)	NO	-
department_id	INTEGER		FK	Departments(department_id)	NO	-
assigned_staff_id	INTEGER		FK	Users(user_id)	YES	-
title	VARCHAR(150)			-	NO	-
description	TEXT			-	NO	-
address	VARCHAR(255)			-	YES	-
latitude	DECIMAL(9,6)			-	YES	-
longitude	DECIMAL(9,6)			-	YES	-
description_embedding	VECTOR(384)			-	YES	-
status	VARCHAR(20)			-	NO	DEFAULT 'Open'; CHECK (status IN ('Open','Assigned','In Progress','Resolved','Closed','Reopened'))
sla_deadline	TIMESTAMP			-	NO	-
created_at	TIMESTAMP			-	NO	DEFAULT CURRENT_TIMESTAMP
updated_at	TIMESTAMP			-	NO	DEFAULT CURRENT_TIMESTAMP
resolved_at	TIMESTAMP			-	YES	-

ComplaintUpvotes

Column	Type	PK	FK	References	Null	Default / Check / Unique
complaint_id	INTEGER	PK	FK	Complaints(complaint_id)	NO	-
citizen_id	INTEGER	PK	FK	Users(user_id)	NO	-
upvoted_at	TIMESTAMP			-	NO	DEFAULT CURRENT_TIMESTAMP

Composite primary key: (complaint_id, citizen_id).

StatusHistory

Column	Type	PK	FK	References	Null	Default / Check / Unique
history_id	SERIAL	PK		-	NO	-
complaint_id	INTEGER		FK	Complaints(complaint_id)	NO	-
old_status	VARCHAR(20)			-	YES	-
new_status	VARCHAR(20)			-	NO	-
changed_by	INTEGER		FK	Users(user_id)	NO	-
remarks	TEXT			-	YES	-
changed_at	TIMESTAMP			-	NO	DEFAULT CURRENT_TIMESTAMP

Escalations

Column	Type	PK	FK	References	Null	Default / Check / Unique
escalation_id	SERIAL	PK		-	NO	-
complaint_id	INTEGER		FK	Complaints(complaint_id)	NO	-
escalated_from	INTEGER		FK	Users(user_id)	YES	-
escalated_to	INTEGER		FK	Users(user_id)	NO	-
escalation_level	INTEGER			-	NO	DEFAULT 1
reason	TEXT			-	NO	-
escalated_at	TIMESTAMP			-	NO	DEFAULT CURRENT_TIMESTAMP
resolved_at	TIMESTAMP			-	YES	-

Notifications

Column	Type	PK	FK	References	Null	Default / Check / Unique
notification_id	SERIAL	PK		-	NO	-
user_id	INTEGER		FK	Users(user_id)	NO	-
complaint_id	INTEGER		FK	Complaints(complaint_id)	YES	-
message	VARCHAR(255)			-	NO	-
type	VARCHAR(30)			-	NO	CHECK (type IN ('assignment','status_update','escalation','duplicate_flag'))
is_read	BOOLEAN			-	NO	DEFAULT FALSE
created_at	TIMESTAMP			-	NO	DEFAULT CURRENT_TIMESTAMP

DuplicateMatches

Column	Type	PK	FK	References	Null	Default / Check / Unique
match_id	SERIAL	PK		-	NO	-
complaint_id	INTEGER		FK	Complaints(complaint_id)	NO	-
matched_complaint_id	INTEGER		FK	Complaints(complaint_id)	NO	-
similarity_score	DECIMAL(5,4)			-	NO	CHECK (similarity_score BETWEEN 0 AND 1)
review_status	VARCHAR(20)			-	NO	DEFAULT 'Pending'; CHECK (review_status IN ('Pending','Confirmed','Rejected'))
detected_at	TIMESTAMP			-	NO	DEFAULT CURRENT_TIMESTAMP

Table-level CHECK: complaint_id <> matched_complaint_id Table-level UNIQUE: (complaint_id, matched_complaint_id).

2. DDL Commands

PostgreSQL DDL implementing the schema above exactly as tabulated — generated directly from the same table definitions, so the two sections cannot drift out of sync.

2.1 Extension & Table Definitions

```
CREATE EXTENSION IF NOT EXISTS vector;
```

Departments

```
CREATE TABLE Departments (
    department_id SERIAL PRIMARY KEY,
    department_name VARCHAR(100) NOT NULL UNIQUE,
    description TEXT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Wards

```
CREATE TABLE Wards (
    ward_id SERIAL PRIMARY KEY,
    ward_name VARCHAR(100) NOT NULL,
    ward_number INTEGER NOT NULL UNIQUE,
    zone VARCHAR(50) NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Users

```
CREATE TABLE Users (
    user_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    phone VARCHAR(15) NOT NULL UNIQUE,
    role VARCHAR(20) NOT NULL CHECK (role IN ('citizen','staff','admin')),
    department_id INTEGER NULL,
    ward_id INTEGER NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CHECK ((role = 'staff' AND department_id IS NOT NULL AND ward_id IS NOT NULL)
        OR (role = 'admin' AND ward_id IS NOT NULL AND department_id IS NULL)
        OR (role = 'citizen' AND department_id IS NULL AND ward_id IS NULL)),
    CONSTRAINT fk_users_department_id FOREIGN KEY (department_id) REFERENCES Departments(department_id),
    CONSTRAINT fk_users_ward_id FOREIGN KEY (ward_id) REFERENCES Wards(ward_id)
);
```

ComplaintCategories

```
CREATE TABLE ComplaintCategories (
    category_id SERIAL PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL UNIQUE,
    department_id INTEGER NOT NULL,
    sla_hours INTEGER NOT NULL DEFAULT 72,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_complaintcategories_department_id FOREIGN KEY (department_id) REFERENCES Departments(department_id)
);
```

Complaints

```
CREATE TABLE Complaints (
    complaint_id SERIAL PRIMARY KEY,
    citizen_id INTEGER NOT NULL,
    category_id INTEGER NOT NULL,
    ward_id INTEGER NOT NULL,
    department_id INTEGER NOT NULL,
    assigned_staff_id INTEGER NULL,
    title VARCHAR(150) NOT NULL,
    description TEXT NOT NULL,
    address VARCHAR(255) NULL,
    latitude DECIMAL(9,6) NULL,
    longitude DECIMAL(9,6) NULL,
    description_embedding VECTOR(384) NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'Open' CHECK (status IN ('Open','Assigned','In Progress','Resolved','Closed','Reopened')),
    sla_deadline TIMESTAMP NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    resolved_at TIMESTAMP NULL,
    CONSTRAINT fk_complaints_citizen_id FOREIGN KEY (citizen_id) REFERENCES Users(user_id),
    CONSTRAINT fk_complaints_category_id FOREIGN KEY (category_id) REFERENCES ComplaintCategories(category_id),
    CONSTRAINT fk_complaints_ward_id FOREIGN KEY (ward_id) REFERENCES Wards(ward_id),
    CONSTRAINT fk_complaints_department_id FOREIGN KEY (department_id) REFERENCES Departments(department_id),
    CONSTRAINT fk_complaints_assigned_staff_id FOREIGN KEY (assigned_staff_id) REFERENCES Users(user_id)
);
```

ComplaintUpvotes

```
CREATE TABLE ComplaintUpvotes (
    complaint_id INTEGER NOT NULL,
    citizen_id INTEGER NOT NULL,
    upvoted_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (complaint_id, citizen_id),
    CONSTRAINT fk_complaintupvotes_complaint_id FOREIGN KEY (complaint_id) REFERENCES Complaints(complaint_id),
    CONSTRAINT fk_complaintupvotes_citizen_id FOREIGN KEY (citizen_id) REFERENCES Users(user_id)
);
```

StatusHistory

```
CREATE TABLE StatusHistory (
    history_id SERIAL PRIMARY KEY,
    complaint_id INTEGER NOT NULL,
    old_status VARCHAR(20) NULL,
    new_status VARCHAR(20) NOT NULL,
    changed_by INTEGER NOT NULL,
    remarks TEXT NULL,
    changed_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_statushistory_complaint_id FOREIGN KEY (complaint_id) REFERENCES Complaints(complaint_id),
    CONSTRAINT fk_statushistory_changed_by FOREIGN KEY (changed_by) REFERENCES Users(user_id)
);
```

Escalations

```
CREATE TABLE Escalations (
    escalation_id SERIAL PRIMARY KEY,
    complaint_id INTEGER NOT NULL,
    escalated_from INTEGER NULL,
    escalated_to INTEGER NOT NULL,
    escalation_level INTEGER NOT NULL DEFAULT 1,
    reason TEXT NOT NULL,
    escalated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    resolved_at TIMESTAMP NULL,
    CONSTRAINT fk_escalations_complaint_id FOREIGN KEY (complaint_id) REFERENCES Complaints(complaint_id),
    CONSTRAINT fk_escalations_escalated_from FOREIGN KEY (escalated_from) REFERENCES Users(user_id),
    CONSTRAINT fk_escalations_escalated_to FOREIGN KEY (escalated_to) REFERENCES Users(user_id)
);
```

Notifications

```
CREATE TABLE Notifications (
    notification_id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL,
    complaint_id INTEGER NULL,
    message VARCHAR(255) NOT NULL,
    type VARCHAR(30) NOT NULL CHECK (type IN ('assignment','status_update','escalation','duplicate_flag')),
    is_read BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_notifications_user_id FOREIGN KEY (user_id) REFERENCES Users(user_id),
    CONSTRAINT fk_notifications_complaint_id FOREIGN KEY (complaint_id) REFERENCES Complaints(complaint_id)
);
```

DuplicateMatches

```
CREATE TABLE DuplicateMatches (
    match_id SERIAL PRIMARY KEY,
    complaint_id INTEGER NOT NULL,
    matched_complaint_id INTEGER NOT NULL,
    similarity_score DECIMAL(5,4) NOT NULL CHECK (similarity_score BETWEEN 0 AND 1),
    review_status VARCHAR(20) NOT NULL DEFAULT 'Pending' CHECK (review_status IN ('Pending','Confirmed','Rejected')),
    detected_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CHECK (complaint_id <> matched_complaint_id),
    UNIQUE (complaint_id, matched_complaint_id),
    CONSTRAINT fk_duplicatematches_complaint_id FOREIGN KEY (complaint_id) REFERENCES Complaints(complaint_id),
    CONSTRAINT fk_duplicatematches_matched_complaint_id FOREIGN KEY (matched_complaint_id) REFERENCES Complaints(complaint_id)
);
```

2.2 Indexes

```
CREATE INDEX idx_complaints_ward ON Complaints(ward_id);
CREATE INDEX idx_complaints_department ON Complaints(department_id);
CREATE INDEX idx_complaints_status ON Complaints(status);
CREATE INDEX idx_complaints_created_at ON Complaints(created_at);
CREATE INDEX idx_complaints_ward_status ON Complaints(ward_id, status);

-- Full-text search index over complaint title + description
CREATE INDEX idx_complaints_fulltext ON Complaints USING GIN (to_tsvector('english', title || ' ' || description));

-- Approximate nearest-neighbour vector index (pgvector) for semantic
-- duplicate detection, using cosine distance
CREATE INDEX idx_complaints_embedding ON Complaints USING ivfflat (description_embedding vector_cosine_ops) WITH (lists = 100);
```

2.3 View

```
CREATE VIEW ward_department_dashboard AS
SELECT
    w.ward_name,
    d.department_name,
    COUNT(*) FILTER (WHERE c.status NOT IN ('Resolved','Closed')) AS open_count,
    COUNT(*) FILTER (WHERE c.status IN ('Resolved','Closed')) AS resolved_count,
    ROUND(
        (AVG(EXTRACT(EPOCH FROM (c.resolved_at - c.created_at)) / 3600.0)
            FILTER (WHERE c.resolved_at IS NOT NULL))::numeric,
        1
    ) AS avg_resolution_hours
FROM Complaints c
JOIN Wards w ON w.ward_id = c.ward_id
JOIN Departments d ON d.department_id = c.department_id
GROUP BY w.ward_name, d.department_name;
```

2.4 Illustrative ALTER / DROP Usage

Demonstrates CREATE / ALTER / DROP as required by the rubric. These are schema-evolution examples layered on top of the base schema above, not part of the initial table definitions.

```
-- Illustrative ALTER usage: schema evolution after initial deployment
ALTER TABLE Complaints ADD COLUMN priority VARCHAR(10) NOT NULL DEFAULT 'Normal'
    CHECK (priority IN ('Low','Normal','High','Critical'));

-- Illustrative ALTER usage: widening a column after a requirements change
ALTER TABLE Users ALTER COLUMN phone TYPE VARCHAR(20);

-- Illustrative DROP usage: removing a deprecated view/index
DROP VIEW IF EXISTS ward_department_dashboard;
DROP INDEX IF EXISTS idx_complaints_ward_status;
```